

Estructuras de Datos y Algoritmos

12 de junio de 2026

Tutoría 3

Autor: *Diego Banda* (diego.banda@mail.udp.cl)

1. Tablas de Hash

1.1. Conceptos básicos

Una *tabla de hash* mapea claves a posiciones mediante una **función hash**. El objetivo es lograr operaciones de **inserción, búsqueda y eliminación en $O(1)$** promedio. Los principales desafíos son el diseño de la función hash y el manejo de colisiones (dos claves con el mismo índice calculado).

Algunas estrategias de resolución de colisiones:

- **Encadenamiento separado (chaining):** Cada posición guarda una lista enlazada con los datos de tal índice.
- **Direccionamiento abierto (open addressing):** Se busca otra posición libre dentro del arreglo (recalcando a través de sondeo lineal, cuadrático, doble hash, etc).

1.2. Implementación de *HashTable* de referencia

```
1 public class HashTable {
2     private int[] table;
3     private int TABLE_SIZE;
4
5     private int hashFunction(int key){
6         return key % TABLE_SIZE;
7     }
8     public HashTable(int TABLE_SIZE) {
9         this.TABLE_SIZE = TABLE_SIZE;
10        table = new int[TABLE_SIZE];
11    }
12    public void insert(int key);
13    public bool search(int key);
14    public void remove(int key);
15 }
```

Listing 1: clase hash de referencia

1.3. Ejercicios

1. Utilizando la función de hash $h(k) = k \% Table_Size$ donde $Table_Size = 11$, inserta los valores:

15	11	27	8	12	5	22	30	1	20
----	----	----	---	----	---	----	----	---	----

- a) ¿Qué pasa si todos los valores tienen el mismo resto? ¿Cómo se llama tal fenómeno?.
 - b) ¿Por qué conviene que $Table_Size$ sea un número primo?.
 - c) Escriba en pseudocódigo las funciones *insert*, *search* y *remove*.
2. El sondeo lineal (o básico) es cuando se tiene una colisión, se prueba guardando en $h+1$, $h+2$, consecutivamente hasta encontrar un espacio disponible.

- a) Agregue a su implementación el sondeo lineal, utilizando $Table_Size = 13$ e inicialmente todas las celdas en -1 (vacías) y -2 las eliminadas.
 - b) ¿Cuál es la importancia del uso diferenciado de -1 y -2 ?
3. Usar doble hashing puede reducir las colisiones.
 - a) A su implementación base (del punto 1) agregue doble hashing a través de las siguientes funciones:

$$h(k, i) = (h_1(k) + i * h_2(k)) \% Table_Size$$

$$h_1(k) = k \% 13$$

$$h_2(k) = MAX(1, (7 - (k \% 7)))$$

4. Utilizando un $Table_Size = 13$, compare las colisiones que ocurren en la implementación de sondeo lineal vs. doble hash insertando los siguientes valores:

10	22	31	4	15	28	17	88	59
----	----	----	---	----	----	----	----	----

2. Grafos

2.1. Conceptos básicos

Un grafo $G = (V, E)$ consiste en un conjunto de vértices V (o nodos) y aristas E (o conexión, camino, etc). Puede ser:

- **Dirigido:** Las aristas tienen dirección.
- **No dirigido:** Las aristas son bidireccionales.
- **Ponderado:** Cada arista tiene un peso o costo asociado.

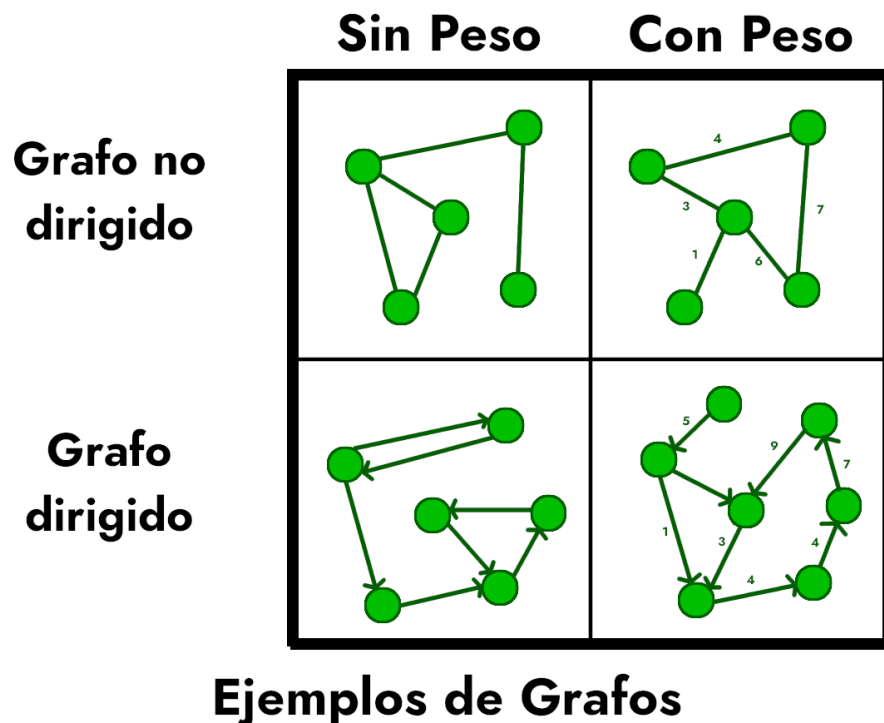


Figura 1: Ejemplos de grafos.

Existen dos representaciones principales:

■ **Matriz de Adyacencia:**

$$adj[i][j] = x$$

Donde:

- i es el vértice desde donde sale la arista, j es el vértice al que llega tal arista.
- si $x = 0$, no existe una arista entre tales vértices.
- si $x = peso/costo$, existe una conexión entre tales aristas (para grafos sin peso $x = 1$).
- Espacio usado: $O(V^2)$.

■ **Lista de Adyacencia:**

$$adj[i] = lista_vecinos$$

Donde:

- i es el vértice desde donde sale la arista, la lista guardada son los vértices a los que llega tal arista.
- Si un vértice no se encuentra en la lista de i , entonces no existe tal arista.
- Para un grafo no ponderado, guarda el número del vértice al que apunta tal vértice. Ejemplo:

$$adj[0] \rightarrow [1, 3, 4, 5]$$

- Para un grafo ponderado, se guarda el par $\{Vértice, Peso/Costo\}$. Ejemplo:

$$adj[0] = [\{1, 4\}, \{3, 3\}, \{4, 10\}, \{5, 2\}]$$

- Espacio usado: $O(V + E)$

2.2. Ejercicios

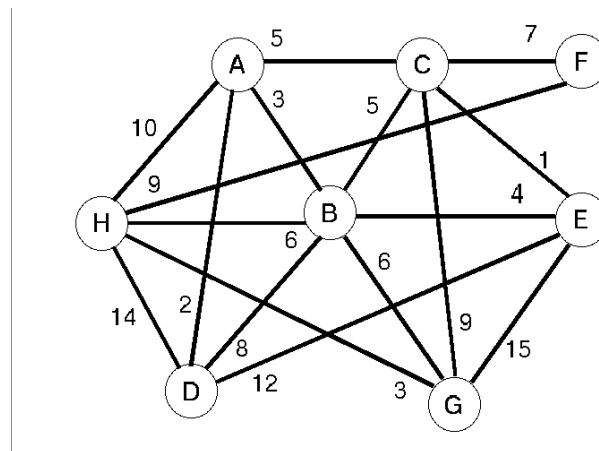


Figura 2: Grafo ponderado no dirigido

1. Para el gráfico de la figura 2 genere la matriz de adyacencia.
2. Para el gráfico de la figura 2 genere la lista de adyacencia.
3. Una búsqueda en anchura (*BFS: Breadth First Search*) es un algoritmo para visitar todos los nodos e imprimirlos, comenzando desde un vértice determinado y visitando primero todos los nodos a los que exista una arista. Ejecute *BFS* para el grafo de la figura 2.

```
1 static ArrayList<Integer> bfs(ArrayList<ArrayList<Integer>> adj) {
2     int V = adj.size();
3     boolean[] visited = new boolean[V];
4     ArrayList<Integer> res = new ArrayList<>();
5
6     int src = 0;
7     Queue<Integer> q = new LinkedList<>();
8     visited[src] = true;
9     q.add(src);
10
11     while (!q.isEmpty()) {
12         int curr = q.poll();
13         res.add(curr);
14
15         for (int x : adj.get(curr)) {
16             if (!visited[x]) {
17                 visited[x] = true;
18                 q.add(x);
19             }
20         }
21     }
22
23     return res;
24 }
```

Listing 2: BFS

4. Una búsqueda en profundidad (*DFS: Depth First Search*) es un algoritmo para visitar todos los nodos e imprimirlos, comenzando desde un vértice determinado y visitando primero todos los nodos hasta el final para luego retroceder y visitar el resto de opciones vecinas. Ejecute *DFS* para el grafo de la figura 2.

```
1 static void dfsRec(ArrayList<ArrayList<Integer>> adj,
2     boolean[] visited, int s, ArrayList<Integer> res)
3 {
4     visited[s] = true;
5     res.add(s);
6
7     for (int i : adj.get(s)) {
8         if (!visited[i]) {
9             dfsRec(adj, visited, i, res);
10        }
11    }
12 }
13
14 static ArrayList<Integer> dfs(ArrayList<ArrayList<Integer>> adj) {
15     boolean[] visited = new boolean[adj.size()];
16     ArrayList<Integer> res = new ArrayList<>();
17     dfsRec(adj, visited, 0, res);
18     return res;
19 }
```

Listing 3: DFS

3. Referencias

- [1] GeeksforGeeks. *Hashing Data Structure*. Recuperado de: <https://www.geeksforgeeks.org/dsa/hashing-data-structure/>

- [2] GeeksforGeeks. *Breadth First Search or BFS for a Graph*. Recuperado de: <https://www.geeksforgeeks.org/dsa/breadth-first-search-or-bfs-for-a-graph/>
- [3] GeeksforGeeks. *Depth First Search or DFS for a Graph*. Recuperado de: <https://www.geeksforgeeks.org/dsa/depth-first-search-or-dfs-for-a-graph/>